

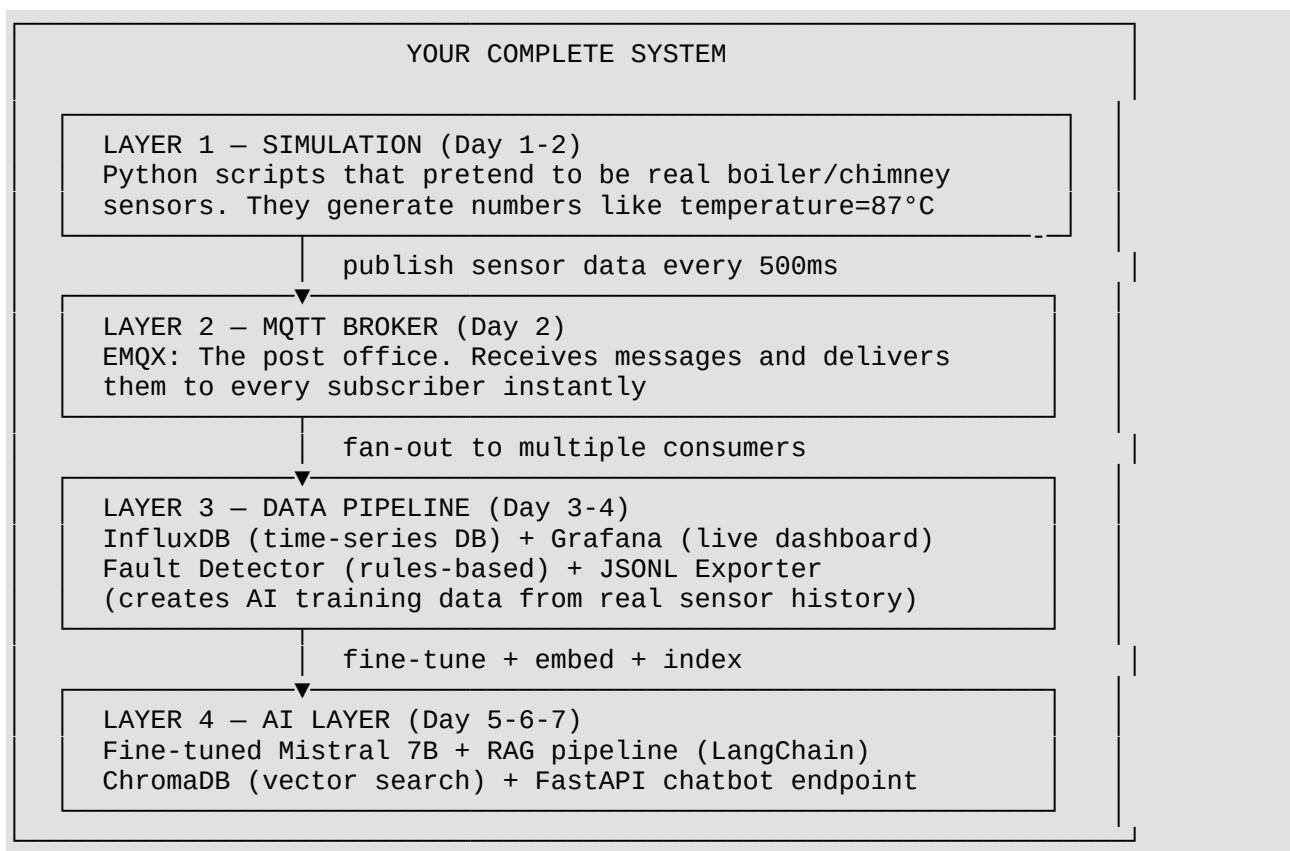
Boiler & Chimney IoT System — 7-Day Build Guide

From Zero to Real-Time AI Chatbot | Fresher-Friendly, Step-by-Step

Who this is for: You are new to IoT and MQTT. This guide assumes you know basic Python and can use a terminal. Every concept is explained before you use it. Every day has a clear goal, all the code you need, and a "what you built today" checklist.

□ System Architecture (Big Picture)

Before writing a single line of code, understand what you are building across all 7 days:



Glossary — Read This First

These are terms you will see every day. Read this once so nothing surprises you.

Term	Plain English Explanation
IoT	Internet of Things — physical devices (sensors, machines) that send data over a network
MQTT	A messaging protocol. Like WhatsApp but for machines. Lightweight, fast, made for sensors

Term	Plain English Explanation
Broker	The MQTT "post office" — receives messages from senders and delivers to receivers
Publisher	A device/script that <i>sends</i> a message to the broker (your boiler simulator)
Subscriber	A device/script that <i>receives</i> messages from the broker (your database writer)
Topic	The "address" of a message. Like boiler/temperature . Publishers and subscribers must use the same topic
QoS	Quality of Service — how reliably a message is delivered (0=fire and forget, 1=at least once, 2=exactly once)
Payload	The actual data inside a message. Example: <code>{"temperature": 87.4, "unit": "C"}</code>
Time-series DB	A database optimised for data that changes over time. Perfect for sensor readings
InfluxDB	A popular time-series database. You will store all sensor data here
Grafana	A tool that reads from InfluxDB and shows live charts on a dashboard
Kafka	A high-speed message queue. Sits between MQTT and your database for 25K+ msg/s
Docker	A tool that runs software in isolated containers. You use it to run EMQX, InfluxDB, Grafana
Fine-tuning	Teaching an existing AI model new domain knowledge using your own data
LoRA	Low Rank Adaptation — a technique to fine-tune a large LLM efficiently on a single GPU
RAG	Retrieval Augmented Generation — before answering, the AI fetches relevant real data and uses it
Vector DB	A database that stores text as mathematical vectors so you can search by meaning, not keywords
ChromaDB	A local vector database — used for RAG retrieval
Embedding	Converting text (like "boiler pressure is high") into a list of numbers that capture its meaning
LangChain	A Python library that connects LLMs, vector DBs, and data sources into one pipeline
FastAPI	A Python web framework for building APIs. You use it to expose your chatbot as an HTTP

Term	Plain English Explanation
	endpoint

7-Day Schedule Overview

Day	Focus	Output
Day 1	Environment setup + MQTT fundamentals	Docker running, EMQX broker live, first publish/subscribe working
Day 2	Boiler & chimney simulator	Python scripts generating realistic sensor data with fault injection
Day 3	InfluxDB + Grafana pipeline	All sensor data stored, live dashboard visible in browser
Day 4	Fault Detector + JSONL Training Data Exporter	Rule-based fault detection running, training dataset generated
Day 5	Fine-tuning the LLM	Mistral 7B fine-tuned on your boiler data using LoRA
Day 6	RAG pipeline + ChromaDB	Chatbot retrieves real-time sensor context before answering
Day 7	FastAPI chatbot + Integration	Full system working end-to-end, chatbot answers from live data

DAY 1 — Environment Setup + MQTT Fundamentals

☐ Goal

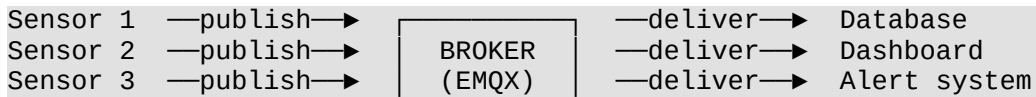
By end of Day 1 you will:

- Have Docker installed and running
- Have EMQX (MQTT broker) running in a container
- Write your first publisher and subscriber in Python
- Understand exactly what MQTT does

Concept: What is MQTT and Why Does It Exist?

Imagine you have 100 temperature sensors on a factory floor. You want all of them to send data to a central server. Without MQTT, each sensor would need to know the server's address and maintain a connection. If the server changes, you update 100 sensors.

MQTT solves this with a **broker in the middle**:



The sensors don't know about the database. The database doesn't know about the sensors. They only know about the broker. This is **decoupling** — the core benefit of pub/sub architecture.

MQTT Topic Structure

Topics are like folder paths. They use / as a separator:

boiler/temperature	← specific sensor
boiler/pressure	← another specific sensor
boiler/#	← wildcard: ALL boiler sensors
chimney/flue_gas/co2	← nested topic
system/faults	← fault events
+/temperature	← wildcard: temperature from ANY device

The # wildcard means "everything below this level." The + wildcard means "any single level."

Step 1.1 — Install Prerequisites

```
# Install Docker Desktop from https://www.docker.com/products/docker-desktop
# Verify it works:
docker --version
docker-compose --version

# Install Python 3.10+
python --version

# Create your project folder
mkdir boiler-iot-system
cd boiler-iot-system

# Create a Python virtual environment
python -m venv venv

# Activate it
# On Windows:
venv\Scripts\activate
# On Mac/Linux:
source venv/bin/activate

# Install Python MQTT library
pip install paho-mqtt
```

Step 1.2 — Run EMQX Broker with Docker

Create a file called `docker-compose.yml` in your project folder:

```
version: '3.8'

services:
  emqx:
    image: emqx/emqx:latest
    container_name: emqx_broker
    ports:
      - "1883:1883"    # MQTT port (your Python scripts connect here)
      - "8083:8083"    # MQTT over WebSocket
      - "18083:18083"  # EMQX web dashboard
    environment:
      - EMQX_ALLOW_ANONYMOUS=true # No password needed for development
    volumes:
      - emqx_data:/opt/emqx/data
    restart: unless-stopped

volumes:
  emqx_data:
```

Start it:

```
docker-compose up -d emqx
```

Open your browser: `http://localhost:18083` Login: `admin / public`

You will see the EMQX dashboard. This is your broker's control panel.

Step 1.3 — Your First Publisher (Hello World)

Create `day1_publisher.py`:

```
"""
Day 1 Publisher – sends a test message to the MQTT broker every second.
Think of this as the boiler sending one temperature reading.
"""

import paho.mqtt.client as mqtt
import json
import time

# ——— CONFIGURATION ———
BROKER_HOST = "localhost"    # EMQX is running on your machine
BROKER_PORT = 1883           # Standard MQTT port
TOPIC = "test/hello"         # The "address" of this message
CLIENT_ID = "publisher_001"  # Unique ID for this client
# ———

def on_connect(client, userdata, flags, rc):
    """Called automatically when connection to broker is established."""
    if rc == 0:
        print("▣ Connected to EMQX broker successfully!")
    else:
        print(f"▣ Connection failed with code: {rc}")

# Create an MQTT client instance
client = mqtt.Client(client_id=CLIENT_ID)
```

```

# Register the callback function
client.on_connect = on_connect

# Connect to the broker
client.connect(BROKER_HOST, BROKER_PORT, keepalive=60)

# Start network loop in background thread
client.loop_start()

# Wait for connection to establish
time.sleep(1)

print("👉 Starting to publish messages...")

message_count = 0
while True:
    message_count += 1

    # Create a JSON payload (real data would come from a sensor)
    payload = {
        "message_id": message_count,
        "text": "Hello from boiler system!",
        "timestamp": time.time()
    }

    # Convert dict to JSON string
    payload_str = json.dumps(payload)

    # Publish to broker
    # QoS=1 means "deliver at least once" (safe for sensor data)
    result = client.publish(TOPIC, payload_str, qos=1)

    print(f"👉 Published message #{message_count}: {payload_str}")

    # Wait 1 second before next message
    time.sleep(1)

```

Step 1.4 — Your First Subscriber

Create `day1_subscriber.py` (run this in a SECOND terminal):

```

"""
Day 1 Subscriber – listens for messages on a topic and prints them.
Think of this as the database consumer receiving boiler data.
"""

import paho.mqtt.client as mqtt
import json

BROKER_HOST = "localhost"
BROKER_PORT = 1883
TOPIC = "test/hello"          # Must match publisher topic
CLIENT_ID = "subscriber_001"

def on_connect(client, userdata, flags, rc):
    """Called when connected to broker."""
    if rc == 0:
        print("👉 Subscriber connected to broker!")
        # Subscribe to topic AFTER connecting

```

```

        client.subscribe(TOPIC, qos=1)
        print(f"👉 Subscribed to topic: {TOPIC}")
    else:
        print(f"❌ Connection failed: {rc}")

def on_message(client, userdata, message):
    """Called AUTOMATICALLY every time a new message arrives on subscribed
    topic."""

    # Decode bytes → string
    payload_str = message.payload.decode("utf-8")

    # Parse JSON string → Python dict
    payload = json.loads(payload_str)

    print(f"👉 Received on [{message.topic}]: {payload}")
    print(f"    Message #{payload['message_id']} arrived at timestamp
    {payload['timestamp']:.2f}")

# Setup client
client = mqtt.Client(client_id=CLIENT_ID)
client.on_connect = on_connect
client.on_message = on_message

# Connect and run forever (blocking)
client.connect(BROKER_HOST, BROKER_PORT, keepalive=60)
print("⌚ Waiting for messages... (Press Ctrl+C to stop)")
client.loop_forever() # This blocks and handles all incoming messages

```

Run both scripts

Terminal 1:

```
python day1_subscriber.py
```

Terminal 2:

```
python day1_publisher.py
```

You will see messages flowing from publisher → broker → subscriber in real time. **This is the entire foundation of your system.**

☐ Day 1 Checklist

- ☐ Docker and docker-compose installed
 - ☐ EMQX running, dashboard accessible at localhost:18083
 - ☐ Publisher script sends messages successfully
 - ☐ Subscriber receives messages in real time
 - ☐ You understand what topic, publish, subscribe, broker mean
-
-

DAY 2 — Boiler & Chimney Simulator with Fault Injection

□ Goal

By end of Day 2 you will have:

- A realistic Python boiler simulator publishing to 8+ MQTT topics
- A chimney simulator running in parallel
- A fault injector that randomly creates equipment problems
- All data in proper JSON format with timestamps

Concept: What Data Does a Real Boiler Generate?

A real industrial boiler has these sensors:

BOILER SENSORS		CHIMNEY SENSORS	
Steam Pressure	→ 10-14 bar	Flue Gas Temp	→ 150-250°C
Water Temperature	→ 60-100°C	CO2 Percentage	→ 8-14%
Fuel Flow Rate	→ 5-20 L/min	O2 Percentage	→ 3-8%
Air Flow Rate	→ 100-300 m/h	CO (Carbon Mono)	→ 0-50 ppm
Drum Water Level	→ 40-60%	Draft Pressure	→ -2 to -5 Pa
Burner State	→ ON/OFF	Stack Velocity	→ 3-8 m/s
Feed Water Flow	→ 5-20 L/min		

Normal ranges are critical — when a value goes outside the normal range, it is a fault.

Step 2.1 — Boiler Simulator

Create `simulators/boiler_simulator.py`:

```
"""
Boiler Simulator – generates realistic boiler sensor data and publishes via
MQTT.
Simulates normal operation + gradual drift + sudden faults.
"""

import paho.mqtt.client as mqtt
import json
import time
import random
import math
from datetime import datetime

# — MQTT CONFIG —
BROKER_HOST = "localhost"
BROKER_PORT = 1883
CLIENT_ID = "boiler_simulator"

# — TOPIC DEFINITIONS —
TOPICS = {
```

```

    "temperature": "boiler/temperature",
    "pressure": "boiler/pressure",
    "fuel_flow": "boiler/fuel_flow",
    "air_flow": "boiler/air_flow",
    "water_level": "boiler/water_level",
    "burner_state": "boiler/burner_state",
    "feed_water": "boiler/feed_water_flow",
    "status": "boiler/status",
    "fault": "system/faults",
}

# ——— NORMAL OPERATING RANGES ———
NORMAL = {
    "temperature": {"min": 60, "max": 100, "unit": "C", "mean": 85},
    "pressure": {"min": 10, "max": 14, "unit": "bar", "mean": 12},
    "fuel_flow": {"min": 5, "max": 20, "unit": "L/min", "mean": 12},
    "air_flow": {"min": 100, "max": 300, "unit": "m3/h", "mean": 200},
    "water_level": {"min": 40, "max": 60, "unit": "%", "mean": 50},
    "feed_water": {"min": 5, "max": 20, "unit": "L/min", "mean": 12},
}

# ——— FAULT DEFINITIONS ———
FAULT_TYPES = {
    "HIGH_PRESSURE": {"sensor": "pressure", "multiplier": 1.3,
"severity": "CRITICAL"},
    "LOW_WATER_LEVEL": {"sensor": "water_level", "multiplier": 0.5,
"severity": "CRITICAL"},
    "HIGH_TEMPERATURE": {"sensor": "temperature", "multiplier": 1.25,
"severity": "WARNING"},
    "LOW_FUEL_FLOW": {"sensor": "fuel_flow", "multiplier": 0.3,
"severity": "WARNING"},
    "BURNER_FAILURE": {"sensor": "burner_state", "multiplier": 0,
"severity": "CRITICAL"},
}

class BoilerSimulator:
    def __init__(self):
        # Current state of all sensors
        self.state = {
            "temperature": 85.0,
            "pressure": 12.0,
            "fuel_flow": 12.0,
            "air_flow": 200.0,
            "water_level": 50.0,
            "burner_state": True,
            "feed_water": 12.0,
        }
        self.active_fault = None
        self.fault_duration = 0
        self.tick = 0 # counts simulation steps

    def _add_noise(self, value, noise_pct=0.02):
        """Add small random noise to simulate sensor jitter."""
        noise = value * noise_pct * random.gauss(0, 1)
        return value + noise

    def _drift_value(self, sensor_name):
        """Make values drift slowly and realistically using sine wave."""
        cfg = NORMAL[sensor_name]
        # Slow oscillation around mean
        drift = math.sin(self.tick * 0.05) * (cfg["max"] - cfg["min"]) * 0.05
        base = cfg["mean"] + drift
        return self._add_noise(base)

```

```

def _inject_fault(self):
    """Randomly decide to start a fault (5% chance per tick)."""
    if self.active_fault is None and random.random() < 0.05:
        fault_name = random.choice(list(FAULT_TYPES.keys()))
        self.active_fault = fault_name
        self.fault_duration = random.randint(10, 30) # lasts 10-30 ticks
        return fault_name
    return None

def update(self):
    """Update all sensor values for this tick."""
    self.tick += 1

    # Normal drift for all sensors
    for sensor in ["temperature", "pressure", "fuel_flow", "air_flow",
"water_level", "feed_water"]:
        self.state[sensor] = round(self._drift_value(sensor), 2)

    # Burner is normally ON
    self.state["burner_state"] = True

    # — Fault logic —————
    new_fault = self._inject_fault()

    if self.active_fault:
        fault = FAULT_TYPES[self.active_fault]
        sensor = fault["sensor"]

        if sensor == "burner_state":
            self.state["burner_state"] = False
        else:
            # Distort the affected sensor value
            normal_val = NORMAL[sensor]["mean"]
            self.state[sensor] = round(normal_val * fault["multiplier"], 2)

        self.fault_duration -= 1
        if self.fault_duration <= 0:
            self.active_fault = None # Fault resolves

    return new_fault # Return fault name if new fault just started

def get_status(self):
    """Return overall health status."""
    if self.active_fault:
        fault = FAULT_TYPES[self.active_fault]
        return fault["severity"]
    return "NORMAL"

def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print("▣ Boiler simulator connected to MQTT broker")
    else:
        print(f"▣ Connection failed: {rc}")

# — MAIN SIMULATION LOOP —————

def main():
    simulator = BoilerSimulator()

    client = mqtt.Client(client_id=CLIENT_ID)
    client.on_connect = on_connect

```

```

client.connect(BROKER_HOST, BROKER_PORT, keepalive=60)
client.loop_start()

time.sleep(1)
print("▣ Boiler simulator started. Publishing every 500ms...")

while True:
    # Update simulation state
    new_fault = simulator.update()

    # Timestamp for all messages this tick
    timestamp = datetime.utcnow().isoformat() + "Z"

    # Publish each sensor to its own topic
    for sensor_name, topic in TOPICS.items():
        if sensor_name in ("status", "fault"):
            continue # Handle separately

        value = simulator.state.get(sensor_name)
        if value is None:
            continue

        payload = {
            "device_id": "BOILER_001",
            "sensor": sensor_name,
            "value": value,
            "unit": NORMAL.get(sensor_name, {}).get("unit", ""),
            "timestamp": timestamp,
            "status": simulator.get_status(),
        }
        client.publish(topic, json.dumps(payload), qos=1)

    # Publish status summary
    status_payload = {
        "device_id": "BOILER_001",
        "timestamp": timestamp,
        "overall_status": simulator.get_status(),
        "active_fault": simulator.active_fault,
        "readings": {k: simulator.state[k] for k in NORMAL},
    }
    client.publish(TOPICS["status"], json.dumps(status_payload), qos=1)

    # Publish fault event if new fault just appeared
    if new_fault:
        fault_payload = {
            "device_id": "BOILER_001",
            "fault_code": new_fault,
            "severity": FAULT_TYPES[new_fault]["severity"],
            "affected_sensor": FAULT_TYPES[new_fault]["sensor"],
            "timestamp": timestamp,
            "message": f"Fault detected: {new_fault} on boiler BOILER_001",
        }
        client.publish(TOPICS["fault"], json.dumps(fault_payload), qos=2)
        print(f"△ FAULT: {new_fault} – severity: {FAULT_TYPES[new_fault]
['severity']}")

    time.sleep(0.5) # 500ms = 2 messages per second per sensor

if __name__ == "__main__":
    main()

```

Step 2.2 — Chimney Simulator

Create `simulators/chimney_simulator.py`:

```
"""
Chimney Simulator – generates flue gas and draft sensor data.
"""

import paho.mqtt.client as mqtt
import json, time, random, math
from datetime import datetime

BROKER_HOST = "localhost"
BROKER_PORT = 1883
CLIENT_ID = "chimney_simulator"

TOPICS = {
    "flue_temp": "chimney/flue_temperature",
    "co2": "chimney/co2_percentage",
    "o2": "chimney/o2_percentage",
    "co": "chimney/co_ppm",
    "draft": "chimney/draft_pressure",
    "stack_velocity": "chimney/stack_velocity",
    "status": "chimney/status",
    "fault": "system/faults",
}

NORMAL = {
    "flue_temp": {"min": 150, "max": 250, "mean": 200, "unit": "C"},
    "co2": {"min": 8, "max": 14, "mean": 11, "unit": "%"},
    "o2": {"min": 3, "max": 8, "mean": 5, "unit": "%"},
    "co": {"min": 0, "max": 50, "mean": 20, "unit": "ppm"},
    "draft": {"min": -5, "max": -2, "mean": -3.5, "unit": "Pa"},
    "stack_velocity": {"min": 3, "max": 8, "mean": 5, "unit": "m/s"},
}

CHIMNEY_FAULTS = {
    "BLOCKED_FLUE": {"sensor": "draft", "effect": "reduce", "factor": 0.3,
"severity": "CRITICAL"},
    "HIGH_CO": {"sensor": "co", "effect": "spike", "factor": 5.0,
"severity": "CRITICAL"},
    "LOW_DRAFT": {"sensor": "draft", "effect": "reduce", "factor": 0.5,
"severity": "WARNING"},
    "HIGH_FLUE_TEMP": {"sensor": "flue_temp", "effect": "spike", "factor": 1.4,
"severity": "WARNING"},
}

class ChimneySimulator:
    def __init__(self):
        self.tick = 0
        self.active_fault = None
        self.fault_duration = 0
        self.state = {k: NORMAL[k]["mean"] for k in NORMAL}

    def update(self):
        self.tick += 1
        for sensor in NORMAL:
            cfg = NORMAL[sensor]
            drift = math.sin(self.tick * 0.04 + 1.5) * (cfg["max"] - cfg["min"])
            * 0.05
            noise = random.gauss(0, 1) * cfg["mean"] * 0.015
            self.state[sensor] = round(cfg["mean"] + drift + noise, 2)
```

```

new_fault = None
if self.active_fault is None and random.random() < 0.04:
    new_fault = random.choice(list(CHIMNEY_FAULTS.keys()))
    self.active_fault = new_fault
    self.fault_duration = random.randint(8, 20)

if self.active_fault:
    fault = CHIMNEY_FAULTS[self.active_fault]
    sensor = fault["sensor"]
    base = NORMAL[sensor]["mean"]
    if fault["effect"] == "spike":
        self.state[sensor] = round(base * fault["factor"], 2)
    elif fault["effect"] == "reduce":
        self.state[sensor] = round(base * fault["factor"], 2)
    self.fault_duration -= 1
    if self.fault_duration <= 0:
        self.active_fault = None

return new_fault

def get_status(self):
    if self.active_fault:
        return CHIMNEY_FAULTS[self.active_fault]["severity"]
    return "NORMAL"

def main():
    sim = ChimneySimulator()
    client = mqtt.Client(client_id=CLIENT_ID)
    client.connect(BROKER_HOST, BROKER_PORT, keepalive=60)
    client.loop_start()
    time.sleep(1)
    print("🏠 Chimney simulator started...")

    while True:
        new_fault = sim.update()
        ts = datetime.utcnow().isoformat() + "Z"

        for sensor_name, topic in TOPICS.items():
            if sensor_name in ("status", "fault"):
                continue
            payload = {
                "device_id": "CHIMNEY_001",
                "sensor": sensor_name,
                "value": sim.state.get(sensor_name),
                "unit": NORMAL.get(sensor_name, {}).get("unit", ""),
                "timestamp": ts,
                "status": sim.get_status(),
            }
            client.publish(topic, json.dumps(payload), qos=1)

        if new_fault:
            fault_payload = {
                "device_id": "CHIMNEY_001",
                "fault_code": new_fault,
                "severity": CHIMNEY_FAULTS[new_fault]["severity"],
                "timestamp": ts,
                "message": f"Chimney fault: {new_fault}",
            }
            client.publish(TOPICS["fault"], json.dumps(fault_payload), qos=2)
            print(f"⚠️ CHIMNEY FAULT: {new_fault}")

        time.sleep(0.5)

```

```
if __name__ == "__main__":  
    main()
```

☐ Day 2 Checklist

- ☐ `boiler_simulator.py` runs and publishes to 8 topics
 - ☐ `chimney_simulator.py` runs and publishes to 7 topics
 - ☐ Fault messages appear occasionally on `system/faults`
 - ☐ You can see messages in EMQX dashboard under Topics tab
 - ☐ Both simulators run simultaneously in separate terminals
-
-

DAY 3 — InfluxDB + Grafana (Live Dashboard)

☐ Goal

By end of Day 3:

- All MQTT sensor data automatically saved to InfluxDB
 - A Grafana dashboard shows live charts of boiler and chimney sensors
 - You can query historical data and see fault events on the timeline
-

Concept: Why InfluxDB Instead of Regular SQL?

A regular SQL database stores rows like:

id	timestamp	sensor_name	value
1	2024-01-01 10:00:00	temperature	85.4
2	2024-01-01 10:00:01	temperature	85.6

InfluxDB stores **measurements** optimised for time queries:

```
measurement: boiler_sensors  
tags: device_id=B0ILER_001, sensor=temperature  
fields: value=85.4  
time: 2024-01-01T10:00:00Z
```

Why it matters: querying "average temperature over the last 5 minutes" on InfluxDB takes milliseconds on millions of rows. On SQL, it can take seconds.

Step 3.1 — Add InfluxDB and Grafana to Docker Compose

Update your `docker-compose.yml`:

```
version: '3.8'

services:
  emqx:
    image: emqx/emqx:latest
    container_name: emqx_broker
    ports:
      - "1883:1883"
      - "8083:8083"
      - "18083:18083"
    environment:
      - EMQX_ALLOW_ANONYMOUS=true
    restart: unless-stopped

  influxdb:
    image: influxdb:2.7
    container_name: influxdb
    ports:
      - "8086:8086"
    environment:
      - DOCKER_INFLUXDB_INIT_MODE=setup
      - DOCKER_INFLUXDB_INIT_USERNAME=admin
      - DOCKER_INFLUXDB_INIT_PASSWORD=password123
      - DOCKER_INFLUXDB_INIT_ORG=boiler_org
      - DOCKER_INFLUXDB_INIT_BUCKET=boiler_data
      - DOCKER_INFLUXDB_INIT_ADMIN_TOKEN=my-super-secret-token-123
    volumes:
      - influxdb_data:/var/lib/influxdb2
    restart: unless-stopped

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_USER=admin
      - GF_SECURITY_ADMIN_PASSWORD=admin123
    volumes:
      - grafana_data:/var/lib/grafana
    depends_on:
      - influxdb
    restart: unless-stopped

volumes:
  influxdb_data:
  grafana_data:

docker-compose up -d
```

Step 3.2 — MQTT to InfluxDB Bridge (Consumer)

Install dependencies:

```
pip install influxdb-client paho-mqtt
```

Create consumers/influx_consumer.py:

```
"""
InfluxDB Consumer – subscribes to ALL MQTT topics and writes every
sensor reading into InfluxDB as a time-series data point.

This is Consumer #1 in your producer/consumer architecture.
"""

import paho.mqtt.client as mqtt
import json
from datetime import datetime
from influxdb_client import InfluxDBClient, Point, WritePrecision
from influxdb_client.client.write_api import SYNCHRONOUS

# — CONFIG —
MQTT_HOST = "localhost"
MQTT_PORT = 1883

INFLUX_URL = "http://localhost:8086"
INFLUX_TOKEN = "my-super-secret-token-123"
INFLUX_ORG = "boiler_org"
INFLUX_BUCKET = "boiler_data"
# —

# Connect to InfluxDB
influx_client = InfluxDBClient(url=INFLUX_URL, token=INFLUX_TOKEN,
org=INFLUX_ORG)
write_api = influx_client.write_api(write_options=SYNCHRONOUS)

print("▢ Connected to InfluxDB")

def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print("▢ InfluxDB consumer connected to MQTT broker")
        # Subscribe to ALL boiler and chimney topics
        client.subscribe("boiler/#", qos=1)
        client.subscribe("chimney/#", qos=1)
        client.subscribe("system/faults", qos=2)
        print("👉 Subscribed to boiler/#, chimney/#, system/faults")

def on_message(client, userdata, message):
    """Every MQTT message → write a data point to InfluxDB."""
    try:
        payload = json.loads(message.payload.decode("utf-8"))
        topic = message.topic

        # Determine measurement name from topic prefix
        if topic.startswith("boiler"):
            measurement = "boiler_sensors"
        elif topic.startswith("chimney"):
            measurement = "chimney_sensors"
        elif topic == "system/faults":
            measurement = "fault_events"
        else:
            return

        device_id = payload.get("device_id", "UNKNOWN")

        # — Write sensor reading —
        if "value" in payload:
            point = (
```

```

        Point(measurement)
        .tag("device_id", device_id)
        .tag("sensor", payload.get("sensor", topic.split("/")[-1]))
        .tag("status", payload.get("status", "NORMAL"))
        .field("value", float(payload["value"]))
        .time(payload.get("timestamp", datetime.utcnow().isoformat()))
    )
    write_api.write(bucket=INFLUX_BUCKET, org=INFLUX_ORG, record=point)
    print(f"▣ Stored: {device_id}/{payload.get('sensor')} = {payload['value']}")

# — Write fault event —————
elif "fault_code" in payload:
    point = (
        Point("fault_events")
        .tag("device_id", device_id)
        .tag("fault_code", payload["fault_code"])
        .tag("severity", payload.get("severity", "UNKNOWN"))
        .field("message", payload.get("message", ""))
        .field("affected_sensor", payload.get("affected_sensor", ""))
        .time(payload.get("timestamp", datetime.utcnow().isoformat()))
    )
    write_api.write(bucket=INFLUX_BUCKET, org=INFLUX_ORG, record=point)
    print(f"▣ Fault stored: {payload['fault_code']} ({payload.get('severity')})")

except Exception as e:
    print(f"▣ Error processing message: {e}")

# Setup MQTT client
mqtt_client = mqtt.Client(client_id="influx_consumer")
mqtt_client.on_connect = on_connect
mqtt_client.on_message = on_message
mqtt_client.connect(MQTT_HOST, MQTT_PORT, keepalive=60)

print("☞ InfluxDB consumer listening...")
mqtt_client.loop_forever()

```

Step 3.3 — Set Up Grafana Dashboard

1. Open <http://localhost:3000> — login: admin / admin123
2. Go to **Configuration** → **Data Sources** → **Add data source**
3. Choose **InfluxDB**
4. Set:
 - Query Language: **Flux**
 - URL: <http://influxdb:8086>
 - Organization: boiler_org
 - Token: my-super-secret-token-123
 - Default Bucket: boiler_data
5. Click **Save & Test** → should say "datasource is working"

Add a panel — Boiler Temperature:

Click + → **New Dashboard** → **Add Panel**

Paste this Flux query:

```
from(bucket: "boiler_data")
  |> range(start: -10m)
  |> filter(fn: (r) => r["_measurement"] == "boiler_sensors")
  |> filter(fn: (r) => r["sensor"] == "temperature")
  |> filter(fn: (r) => r["_field"] == "value")
```

Set refresh to 5s and you will see live temperature data.

☐ Day 3 Checklist

- ☐ InfluxDB running, accessible at localhost:8086
 - ☐ Grafana running, accessible at localhost:3000
 - ☐ `influx_consumer.py` running and storing data
 - ☐ Grafana dashboard shows live temperature chart
 - ☐ Fault events visible in InfluxDB
-
-

DAY 4 — Fault Detector + JSONL Training Data Exporter

☐ Goal

By end of Day 4:

- A rule-based fault detector subscribes to MQTT and catches anomalies in real time
 - An alert logger writes all detected faults into InfluxDB with severity and context
 - A JSONL exporter reads the stored history and generates fine-tuning training data
 - You have a proper `train.jsonl` file ready for Day 5 LLM fine-tuning
-

Concept: What is a Fault Detector?

Your simulators already inject faults occasionally — but right now nothing is *catching* them intelligently. The InfluxDB consumer just blindly stores every value. You need a separate consumer whose only job is to watch values in real time and shout when something is wrong.

A **rule-based fault detector** works like this:

```
Incoming message: boiler/pressure → value = 18.2 bar
```

▼
Check rule: pressure > 14 bar?

YES → CRITICAL FAULT

▼
Publish to: system/faults

Write to: InfluxDB fault_events

Print: ⚠ CRITICAL: HIGH_PRESSURE – 18.2 bar (limit: 14 bar)

This is **Consumer #2** in your architecture. It runs alongside the InfluxDB consumer, both subscribed to the same MQTT topics simultaneously — EMQX delivers the same message to both.

EMQX Broker

```
├── Consumer 1: influx_consumer.py    (stores everything)
├── Consumer 2: fault_detector.py      (watches for anomalies) ← Day 4
└── Consumer 3: rag_indexer.py        (vector DB – Day 6)
```

Concept: What is JSONL and Why Does the LLM Need It?

JSONL = **JSON Lines** — a file where every single line is a complete, valid JSON object.

Your LLM does not learn from raw sensor numbers. It learns from **question-answer pairs** that teach it:

- How to interpret a sensor reading
- What a fault means in plain English
- How to say "yes this is dangerous" vs "this is normal"

Each line in `train.jsonl` looks like this:

```
{"messages": [{"role": "user", "content": "What is the boiler pressure?"}, {"role": "assistant", "content": "Boiler pressure is 18.2 bar, which is ABOVE the safe limit of 14 bar. This is a HIGH_PRESSURE fault – immediate inspection required."}]}
```

The JSONL exporter reads your InfluxDB history and automatically generates hundreds of these pairs. The more hours your simulator has been running, the more diverse your dataset.

Step 4.1 — Fault Detector Consumer

Create `consumers/fault_detector.py`:

```
"""
Fault Detector – Consumer #2.
Subscribes to all MQTT sensor topics, applies threshold rules,
and raises fault alerts when values go out of range.

Runs independently alongside influx_consumer.py.
"""

import paho.mqtt.client as mqtt
from influxdb_client import InfluxDBClient, Point
```

```

from influxdb_client.client.write_api import SYNCHRONOUS
import json
from datetime import datetime

# --- CONFIG ---
MQTT_HOST      = "localhost"
MQTT_PORT      = 1883
INFLUX_URL     = "http://localhost:8086"
INFLUX_TOKEN   = "my-super-secret-token-123"
INFLUX_ORG     = "boiler_org"
INFLUX_BUCKET  = "boiler_data"

# --- THRESHOLD RULES ---
# Format: sensor_name → (min_ok, max_ok, severity, fault_code, unit)
BOILER_RULES = {
    "temperature": (60, 100, "WARNING", "HIGH_TEMP", "°C"),
    "pressure":    (10, 14, "CRITICAL", "HIGH_PRESSURE", "bar"),
    "water_level": (40, 60, "CRITICAL", "ABNORMAL_LEVEL", "%"),
    "fuel_flow":   (5, 20, "WARNING", "ABNORMAL_FUEL", "L/min"),
    "air_flow":    (100, 300, "WARNING", "ABNORMAL_AIRFLOW", "m3/h"),
}

CHIMNEY_RULES = {
    "flue_temp":   (150, 250, "WARNING", "HIGH_FLUE_TEMP", "°C"),
    "co2":          (8, 14, "WARNING", "ABNORMAL_CO2", "%"),
    "co":           (0, 50, "CRITICAL", "HIGH_CO", "ppm"),
    "draft":        (-5, -2, "WARNING", "ABNORMAL_DRAFT", "Pa"),
    "stack_velocity": (3, 8, "WARNING", "ABNORMAL_STACK", "m/s"),
}

# InfluxDB write client
influx_client = InfluxDBClient(url=INFLUX_URL, token=INFLUX_TOKEN,
                                org=INFLUX_ORG)
write_api = influx_client.write_api(write_options=SYNCHRONOUS)

# Track recent faults to avoid duplicate alerts (cooldown per sensor)
fault_cooldown = {} # {sensor_key: last_fault_timestamp}
COOLDOWN_SECONDS = 30

def check_threshold(device_id: str, sensor: str, value: float, rules: dict) -> dict | None:
    """
    Check if a sensor value violates a threshold rule.
    Returns a fault dict if violated, None if normal.
    """
    if sensor not in rules:
        return None

    min_ok, max_ok, severity, fault_code, unit = rules[sensor]

    # Value is within normal range – no fault
    if min_ok <= value <= max_ok:
        return None

    # Check cooldown – don't spam the same fault every 500ms
    cooldown_key = f"{device_id}_{sensor}"
    now = datetime.utcnow().timestamp()
    last_fault_time = fault_cooldown.get(cooldown_key, 0)

    if now - last_fault_time < COOLDOWN_SECONDS:
        return None # Still in cooldown, skip

```

```

# Record this fault time
fault_cooldown[cooldown_key] = now

# Determine direction
if value > max_ok:
    direction = f"above maximum ({max_ok} {unit})"
else:
    direction = f"below minimum ({min_ok} {unit})"

return {
    "device_id":    device_id,
    "fault_code":   fault_code,
    "severity":     severity,
    "sensor":       sensor,
    "value":        value,
    "unit":         unit,
    "normal_min":   min_ok,
    "normal_max":   max_ok,
    "direction":    direction,
    "timestamp":    datetime.utcnow().isoformat() + "Z",
    "message": (
        f"{severity}: {fault_code} on {device_id}. "
        f"{sensor} = {value} {unit} is {direction}. "
        f"Normal range: {min_ok}-{max_ok} {unit}."
    ),
}

```

```

def handle_fault(client, fault: dict):
    """Log fault to InfluxDB and publish alert to MQTT."""

    # — Write to InfluxDB —————
    point = (
        Point("fault_events")
        .tag("device_id", fault["device_id"])
        .tag("fault_code", fault["fault_code"])
        .tag("severity", fault["severity"])
        .tag("sensor", fault["sensor"])
        .field("value", float(fault["value"]))
        .field("normal_min", float(fault["normal_min"]))
        .field("normal_max", float(fault["normal_max"]))
        .field("message", fault["message"])
        .field("affected_sensor", fault["sensor"])
        .time(fault["timestamp"])
    )
    write_api.write(bucket=INFLUX_BUCKET, org=INFLUX_ORG, record=point)

    # — Publish fault alert back to MQTT —————
    # Other consumers (e.g. a notification service) can subscribe to
    system/faults
    client.publish("system/faults", json.dumps(fault), qos=2)

    # — Console log —————
    emoji = "⚠️"
    print(f"{emoji} {fault['message']}")

    """if fault['severity'] == "CRITICAL":
        emoji = "🔥"""

def on_connect(mqtt_client, userdata, flags, rc):
    if rc == 0:
        mqtt_client.subscribe("boiler/#", qos=1)
        mqtt_client.subscribe("chimney/#", qos=1)
        print("▯ Fault detector connected – watching all sensors")

```

```

def on_message(mqtt_client, userdata, message):
    """Process every incoming sensor message through threshold rules."""
    try:
        payload = json.loads(message.payload.decode("utf-8"))
        topic = message.topic

        # Only process individual sensor readings (not status summaries)
        if "value" not in payload:
            return

        device_id = payload.get("device_id", "UNKNOWN")
        sensor_name = payload.get("sensor", "")
        value = float(payload["value"])

        # Pick the right ruleset based on topic prefix
        if topic.startswith("boiler"):
            fault = check_threshold(device_id, sensor_name, value, BOILER_RULES)
        elif topic.startswith("chimney"):
            fault = check_threshold(device_id, sensor_name, value,
CHIMNEY_RULES)
        else:
            return

        if fault:
            handle_fault(mqtt_client, fault)

    except Exception as e:
        print(f" Fault detector error: {e}")

# ——— START ———
client = mqtt.Client(client_id="fault_detector")
client.on_connect = on_connect
client.on_message = on_message
client.connect(MQTT_HOST, MQTT_PORT, keepalive=60)

print(" Fault detector started. Monitoring all thresholds...")
client.loop_forever()

```

Run it in a new terminal:

```
python consumers/fault_detector.py
```

You will start seeing fault alerts whenever the simulators inject anomalies. Both `influx_consumer.py` and `fault_detector.py` run at the same time — they each independently receive every MQTT message.

Step 4.2 — Verify Faults in InfluxDB

Check that faults are being stored. Open the InfluxDB UI at <http://localhost:8086>, go to **Data Explorer**, and run:

```

from(bucket: "boiler_data")
  |> range(start: -1h)
  |> filter(fn: (r) => r["_measurement"] == "fault_events")
  |> filter(fn: (r) => r["_field"] == "message")

```

You should see fault messages appearing. Add a Grafana panel for this too — use a **Table** visualization to show faults as a list.

Step 4.3 — JSONL Training Data Exporter

Now that you have real sensor history AND fault history in InfluxDB, you can generate training data. This script reads both and creates diverse Q&A pairs.

Install dependency (already installed from Day 3):

```
pip install influxdb-client
```

Create exporters/jsonl_exporter.py:

```
"""
JSONL Exporter – reads sensor history and fault events from InfluxDB,
then generates question-answer pairs for LLM fine-tuning.

Run this after the simulators have been running for at least 1-2 hours
so you have enough diverse data to generate meaningful training pairs.

Output: data/train.jsonl – one JSON object per line, ChatML format.
"""

import json, os
from datetime import datetime
from influxdb_client import InfluxDBClient

INFLUX_URL = "http://localhost:8086"
INFLUX_TOKEN = "my-super-secret-token-123"
INFLUX_ORG = "boiler_org"
INFLUX_BUCKET = "boiler_data"
OUTPUT_FILE = "data/train.jsonl"

client = InfluxDBClient(url=INFLUX_URL, token=INFLUX_TOKEN, org=INFLUX_ORG)
query_api = client.query_api()

# ——— SENSOR METADATA ———
SENSOR_META = {
    # boiler
    "temperature": {"min": 60, "max": 100, "unit": "°C", "device":
"BOILER_001", "measurement": "boiler_sensors"},
    "pressure": {"min": 10, "max": 14, "unit": "bar", "device":
"BOILER_001", "measurement": "boiler_sensors"},
    "fuel_flow": {"min": 5, "max": 20, "unit": "L/min", "device":
"BOILER_001", "measurement": "boiler_sensors"},
    "water_level": {"min": 40, "max": 60, "unit": "%", "device":
"BOILER_001", "measurement": "boiler_sensors"},
    # chimney
    "flue_temp": {"min": 150, "max": 250, "unit": "°C", "device":
"CHIMNEY_001", "measurement": "chimney_sensors"},
    "co2": {"min": 8, "max": 14, "unit": "%", "device":
"CHIMNEY_001", "measurement": "chimney_sensors"},
    "co": {"min": 0, "max": 50, "unit": "ppm", "device":
"CHIMNEY_001", "measurement": "chimney_sensors"},
    "draft": {"min": -5, "max": -2, "unit": "Pa", "device":
"CHIMNEY_001", "measurement": "chimney_sensors"},
}
```

```

def get_avg(sensor: str, minutes: int = 60) -> float | None:
    """Get average sensor value over the last N minutes."""
    meta = SENSOR_META[sensor]
    query = f"""
    from(bucket: "{INFLUX_BUCKET}")
    |> range(start: -{minutes}m)
    |> filter(fn: (r) => r["_measurement"] == "{meta['measurement']}")
    |> filter(fn: (r) => r["sensor"] == "{sensor}")
    |> filter(fn: (r) => r["_field"] == "value")
    |> mean()
    """
    try:
        tables = query_api.query(query)
        for table in tables:
            for record in table.records:
                return round(record.get_value(), 2)
    except Exception:
        pass
    return None

def get_fault_history(hours: int = 24) -> list:
    """Get all fault events in the last N hours."""
    query = f"""
    from(bucket: "{INFLUX_BUCKET}")
    |> range(start: -{hours}h)
    |> filter(fn: (r) => r["_measurement"] == "fault_events")
    |> filter(fn: (r) => r["_field"] == "message")
    |> sort(columns: ["_time"], desc: true)
    """
    faults = []
    try:
        tables = query_api.query(query)
        for table in tables:
            for record in table.records:
                faults.append({
                    "time": str(record.get_time()),
                    "fault_code": record.values.get("fault_code", "UNKNOWN"),
                    "severity": record.values.get("severity", "UNKNOWN"),
                    "sensor": record.values.get("sensor", ""),
                    "message": record.get_value(),
                })
    except Exception:
        pass
    return faults

def qa(user_msg: str, assistant_msg: str) -> dict:
    """Helper to build a ChatML-format Q&A pair."""
    return {
        "messages": [
            {"role": "user", "content": user_msg},
            {"role": "assistant", "content": assistant_msg},
        ]
    }

def generate_pairs() -> list:
    pairs = []

    # — 1. Individual sensor reading pairs —————
    for sensor, meta in SENSOR_META.items():
        value = get_avg(sensor)
        if value is None:

```

```

        continue

        lo, hi, unit = meta["min"], meta["max"], meta["unit"]
        device = meta["device"]
        name = sensor.replace("_", " ")

        in_range = lo <= value <= hi
        status_str = "within the normal range" if in_range else "OUTSIDE the
normal range – attention required"

        pairs.append(qa(
            f"What is the current {name}?",
            f"The current {name} on {device} is {value} {unit}, which is
{status_str} (normal: {lo}-{hi} {unit})."
        ))
        pairs.append(qa(
            f"Is the {name} reading safe?",
            f"The {name} is currently {value} {unit}. The safe operating range
is {lo}-{hi} {unit}. "
            + ("Reading is safe." if in_range else f"△ Reading is unsafe –
{value} {unit} is {'above' if value > hi else 'below'} the limit.")
        ))
        pairs.append(qa(
            f"What is the normal range for {name}?",
            f"The normal operating range for {name} on {device} is {lo}-{hi}
{unit}."
        ))

# — 2. Overall system status pair —————
all_ok = True
summary_parts = []
for sensor, meta in SENSOR_META.items():
    value = get_avg(sensor)
    if value is None:
        continue
    lo, hi, unit = meta["min"], meta["max"], meta["unit"]
    ok = lo <= value <= hi
    if not ok:
        all_ok = False
    summary_parts.append(f"{sensor}={value}{unit}({'OK' if ok else
'FAULT'})")

if summary_parts:
    summary = ", ".join(summary_parts)
    pairs.append(qa(
        "Give me a full system status report.",
        f"System status: {'ALL NORMAL ▯' if all_ok else 'FAULTS DETECTED
△'}. "
        f"Current readings: {summary}."
    ))
    pairs.append(qa(
        "Is the boiler safe to operate right now?",
        f"{'Yes, all sensor readings are within normal operating limits. The
boiler is safe to operate.' if all_ok else 'No, one or more sensors are outside
normal limits. Review faults before operating.'} "
        f"Summary: {summary}."
    ))

# — 3. Fault history pairs —————
faults = get_fault_history(hours=24)

if faults:
    fault_lines = "\n".join([
        f"- [{f['severity']}] {f['fault_code']} on sensor '{f['sensor']}' at

```

```

{f['time']}]}"
    for f in faults[:10]
    ])
    pairs.append(qa(
        "What faults occurred in the last 24 hours?",
        f"{len(faults)} fault event(s) recorded in the last 24 hours:
\n{fault_lines}"
    ))

    critical = [f for f in faults if f["severity"] == "CRITICAL"]
    if critical:
        crit_lines = "\n".join([f"- {f['fault_code']}: {f['message']}" for f
in critical[:5]])
        pairs.append(qa(
            "Are there any critical faults?",
            f"Yes, {len(critical)} critical fault(s) detected:\n{crit_lines}
\nImmediate inspection required."
        ))
    else:
        pairs.append(qa(
            "Are there any critical faults?",
            "No critical faults in the last 24 hours. Some warnings may
exist – check the fault log for details."
        ))

    # Most recent fault
    latest = faults[0]
    pairs.append(qa(
        "What was the most recent fault?",
        f"Most recent fault: [{latest['severity']}] {latest['fault_code']}
on sensor "
        f"'{latest['sensor']}' at {latest['time']}. Details:
{latest['message']}"
    ))

    else:
        pairs.append(qa(
            "Are there any active faults?",
            "No faults detected in the last 24 hours. All boiler and chimney
systems are operating normally."
        ))
        pairs.append(qa(
            "What faults occurred in the last 24 hours?",
            "No fault events were recorded in the last 24 hours. The system has
been operating within normal parameters."
        ))

    # — 4. Fault explanation pairs (static domain knowledge) —
    fault_explanations = {
        "HIGH_PRESSURE": (
            "HIGH_PRESSURE occurs when boiler steam pressure exceeds 14 bar. "
            "Causes: faulty pressure relief valve, excessive heat input, blocked
steam outlet. "
            "Action: reduce burner output immediately and inspect pressure
relief valve."
        ),
        "LOW_WATER_LEVEL": (
            "LOW_WATER_LEVEL means drum water level dropped below 40%. "
            "This is a critical condition – running a boiler dry causes
overheating and tube failure. "
            "Action: shut down burner immediately and restore water supply."
        ),
        "HIGH_CO": (
            "HIGH_CO (carbon monoxide above 50 ppm) in chimney flue gas

```

```

indicates incomplete combustion. "
    "Causes: insufficient air supply, burner misalignment, blocked flue.
"
    "Action: check air-to-fuel ratio and inspect burner nozzle."
),
"BLOCKED_FLUE": (
    "BLOCKED_FLUE means draft pressure is abnormally low (less negative
than -5 Pa). "
    "The chimney cannot draw flue gases out properly. "
    "Causes: soot buildup, bird nests, mechanical obstruction. "
    "Action: shut down and inspect chimney flue physically."
),
"HIGH_FLUE_TEMP": (
    "HIGH_FLUE_TEMP means chimney outlet temperature exceeded 250°C. "
    "This indicates poor heat transfer in the heat exchanger. "
    "Causes: scale buildup on tubes, high excess air. "
    "Action: schedule descaling maintenance."
),
}

for fault_code, explanation in fault_explanations.items():
    pairs.append(qa(
        f"What does the {fault_code} fault mean?",
        explanation
    ))
    pairs.append(qa(
        f"How do I fix a {fault_code} fault?",
        explanation
    ))

return pairs

# ——— MAIN ———
os.makedirs("data", exist_ok=True)

print("\n Reading InfluxDB and generating JSONL training pairs...")
pairs = generate_pairs()

with open(OUTPUT_FILE, "w") as f:
    for pair in pairs:
        f.write(json.dumps(pair) + "\n")

print(f"▣ Exported {len(pairs)} Q&A pairs → {OUTPUT_FILE}")
print(f"    Run the simulators for longer to get more diverse data, then re-run
this script.")

```

Run it:

```
python exporters/jsonl_exporter.py
```

Example output:

```

▣ Reading InfluxDB and generating JSONL training pairs...
▣ Exported 47 Q&A pairs → data/train.jsonl
    Run the simulators for longer to get more diverse data, then re-run this
script.

```

Tip: Let the simulators run for 2–3 hours before generating your dataset. The fault injectors will have triggered 10–20 different fault events by then, giving you much richer training examples. Re-run the exporter as many times as you want — each run reflects the latest data.

Step 4.4 — Inspect Your Training Data

```
# See how many lines (= training pairs) you have
wc -l data/train.jsonl

# Preview first 3 pairs nicely
head -3 data/train.jsonl | python -m json.tool
```

A healthy `train.jsonl` for Day 5 has at least **80–100 pairs**. If you have fewer, keep the simulators running and re-export.

☐ Day 4 Checklist

- ☐ `fault_detector.py` running and printing fault alerts in terminal
 - ☐ Fault events visible in InfluxDB (`fault_events` measurement)
 - ☐ Grafana table panel showing recent faults
 - ☐ `jsonl_exporter.py` generates `data/train.jsonl` successfully
 - ☐ File has 80+ Q&A pairs
 - ☐ You understand what each Q&A pair teaches the LLM
-

☐ TO-DO (Future / Production)

The current Day 4 implementation uses direct MQTT fan-out for multiple consumers — which is perfectly sufficient for this project. When you scale to hundreds of real physical devices (25,000+ messages/second), replace the direct MQTT → consumer pattern with this production-grade buffer:

```
MQTT Broker → Kafka Topics → Consumer Group Workers
├── boiler-sensors  topic → InfluxDB writer worker(s)
├── chimney-sensors topic → Fault detector worker(s)
└── system-faults  topic → Alert + RAG indexer worker(s)
```

Why Kafka at scale:

- Consumers fall behind? Kafka buffers the backlog on disk — zero message loss
- InfluxDB goes down for 2 hours? Replay from Kafka when it comes back
- Need 10× throughput? Add more consumer workers to the same consumer group
- Audit trail? Kafka retains all messages for 7 days by default

Stack to add: `confluentinc/cp-kafka` Docker image + `kafka-python` library + one bridge script (`consumers/kafka_bridge.py`) to forward MQTT → Kafka topics.

DAY 5 — Fine-Tuning the LLM with LoRA

□ Goal

Fine-tune Mistral 7B or Llama 3.1 8B on your boiler training data using LoRA, so the model understands your domain.

Concept: Fine-Tuning vs RAG — When to Use Each

	Fine-Tuning	RAG
What it does	Bakes domain knowledge into model weights	Fetches relevant context at query time
Good for	Domain vocabulary, fault code definitions, equipment-specific reasoning	Real-time data, latest readings, recent faults
Updates	Requires retraining	Immediate — just update the vector DB
You need both	Fine-tune for <i>how to reason</i> about boiler data	RAG for <i>what the current data says</i>

You are building a **hybrid system**: fine-tuning teaches the model what CO₂ at 18% means for a chimney; RAG tells it that CO₂ right now is 18%.

Concept: LoRA — Low Rank Adaptation

A 7B parameter model has 7 billion numbers in its weights. Full fine-tuning changes all of them — requires 8+ GPUs.

LoRA adds two small matrices (A and B) next to each weight matrix. Only A and B are trained. This is ~0.1% of the total parameters.

```
Original weight W (frozen)
      +
LoRA adapter: A × B (trained)
      =
Effective weight: W + A×B
```

You can fine-tune a 7B model on a single RTX 3090 / A100 GPU in 2-4 hours.

Step 5.1 — Install Fine-Tuning Dependencies

```
pip install transformers peft datasets accelerate bitsandbytes torch
```

Create `training/fine_tune.py`:

```

"""
Fine-tuning script using LoRA (QLoRA for 4-bit quantization).
Trains Mistral 7B on your boiler Q&A dataset.

Requirements:
- GPU with 16GB+ VRAM (RTX 3090 / A100 / T4)
- Or use Google Colab A100 (free tier available)
"""

import json
import torch
from datasets import Dataset
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    TrainingArguments,
    Trainer,
    BitsAndBytesConfig,
)
from peft import get_peft_model, LoraConfig, TaskType,
prepare_model_for_kbit_training

# — CONFIG —————
BASE_MODEL = "mistralai/Mistral-7B-Instruct-v0.2" # Or "meta-llama/Llama-3.1-8B-Instruct"
TRAIN_FILE = "../data/train.jsonl"
OUTPUT_DIR = "../models/boiler_llm"
MAX_LENGTH = 512

# LoRA hyperparameters
LORA_R = 16 # Rank — higher = more capacity but more memory
LORA_ALPHA = 32 # Scaling factor (usually 2x rank)
LORA_DROPOUT = 0.05
# —————

def load_jsonl(filepath):
    """Load JSONL file into a list of dicts."""
    records = []
    with open(filepath) as f:
        for line in f:
            records.append(json.loads(line.strip()))
    return records

def format_prompt(example):
    """Convert ChatML format to model's expected format."""
    messages = example["messages"]
    user_msg = next(m["content"] for m in messages if m["role"] == "user")
    asst_msg = next(m["content"] for m in messages if m["role"] == "assistant")

    # Mistral instruction format
    text = f"<s>[INST] {user_msg} [/INST] {asst_msg} </s>"
    return {"text": text}

# — 1. Load and prepare dataset —————
print("▣ Loading training data...")
raw_data = load_jsonl(TRAIN_FILE)
dataset = Dataset.from_list(raw_data).map(format_prompt)
dataset = dataset.train_test_split(test_size=0.1)
print(f"▣ Dataset: {len(dataset['train'])} train, {len(dataset['test'])} eval samples")

```

```

# — 2. Load tokenizer —————
print("▣ Loading tokenizer...")
tokenizer = AutoTokenizer.from_pretrained(BASE_MODEL, trust_remote_code=True)
tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = "right"

# — 3. Load model in 4-bit (QLoRA) —————
print("▣ Loading model in 4-bit quantization (QLoRA)...")
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
)

model = AutoModelForCausalLM.from_pretrained(
    BASE_MODEL,
    quantization_config=bnb_config,
    device_map="auto",
    trust_remote_code=True,
)
model = prepare_model_for_kbit_training(model)

# — 4. Configure LoRA —————
print("▣ Applying LoRA adapters...")
lora_config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=LORA_R,
    lora_alpha=LORA_ALPHA,
    lora_dropout=LORA_DROPOUT,
    target_modules=["q_proj", "v_proj", "k_proj", "o_proj"], # Attention
    matrices
    bias="none",
)
model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# Output: trainable params: 4,194,304 || all params: 3,756,040,192 ||
# trainable%: 0.11%

# — 5. Tokenize —————
def tokenize(example):
    return tokenizer(
        example["text"],
        truncation=True,
        max_length=MAX_LENGTH,
        padding="max_length",
    )

tokenized = dataset.map(tokenize, batched=True)

# — 6. Training arguments —————
training_args = TrainingArguments(
    output_dir=OUTPUT_DIR,
    num_train_epochs=3,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4, # Effective batch = 4 × 4 = 16
    learning_rate=2e-4,
    fp16=True,
    save_steps=100,
    logging_steps=25,
    evaluation_strategy="steps",
    eval_steps=100,
    warmup_steps=50,
    report_to="none",

```

```

)

# — 7. Train —————
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized["train"],
    eval_dataset=tokenized["test"],
)

print("▣ Starting fine-tuning...")
trainer.train()
model.save_pretrained(OUTPUT_DIR)
tokenizer.save_pretrained(OUTPUT_DIR)
print(f"▣ Fine-tuned model saved to {OUTPUT_DIR}")

```

No GPU? Upload `train.jsonl` to Google Colab and run there. Free T4 is sufficient for this dataset size.

☐ Day 5 Checklist

- ☐ Fine-tuning dependencies installed
- ☐ `train.jsonl` has 100+ Q&A pairs
- ☐ Fine-tuning runs without OOM errors
- ☐ Model saved to `models/boiler_llm/`

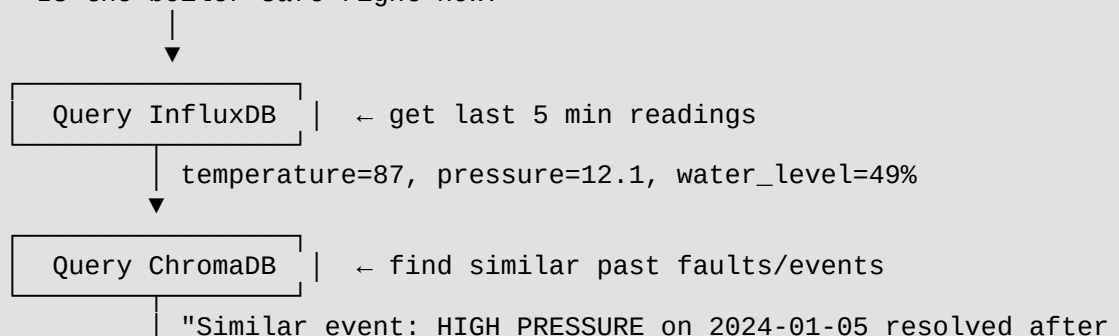
DAY 6 — RAG Pipeline with ChromaDB + LangChain

☐ Goal

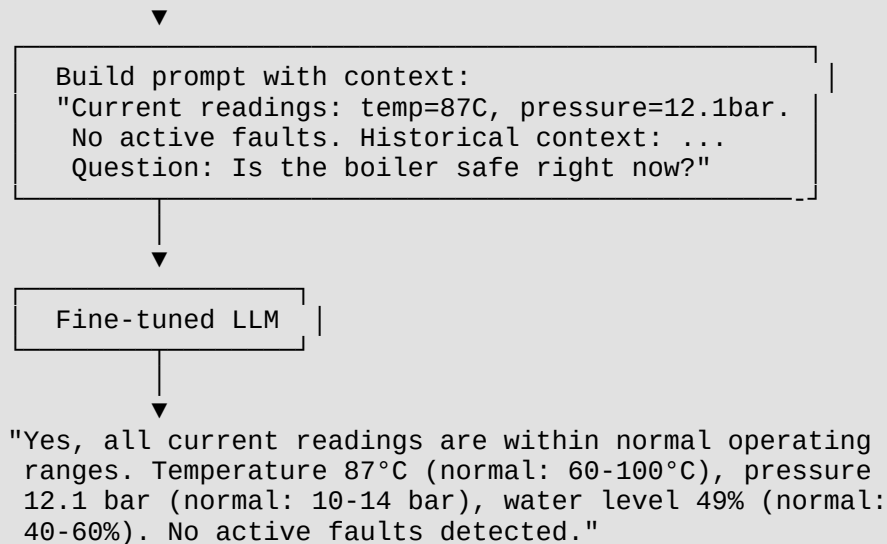
Build a RAG pipeline that fetches the latest sensor readings before every LLM query, so the chatbot always answers from real current data.

Concept: How RAG Works in This System

User asks: "Is the boiler safe right now?"



8 min"



Step 6.1 — Install RAG Dependencies

```
pip install langchain langchain-community chromadb sentence-transformers ollama
```

Serve your fine-tuned model with Ollama:

```
# Install Ollama from https://ollama.ai
# Then create a Modelfile
cat > Modelfile << EOF
FROM ./models/boiler_llm
SYSTEM "You are a boiler and chimney monitoring assistant. You only answer
questions based on real-time sensor data and fault history provided to you. Do
not use general knowledge."
EOF

ollama create boiler-assistant -f Modelfile
ollama serve # starts model server on localhost:11434
```

Step 6.2 — RAG Pipeline

Create rag/rag_pipeline.py:

```
"""
RAG Pipeline – connects InfluxDB (real-time data), ChromaDB (semantic search),
and the fine-tuned LLM to answer boiler/chimney questions grounded in real data.
"""

from langchain.llms import Ollama
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import Chroma
from langchain.prompts import PromptTemplate
from langchain.chains import RetrievalQA
from influxdb_client import InfluxDBClient
import chromadb
import json
from datetime import datetime
```

```

# — CONFIG —
INFLUX_URL      = "http://localhost:8086"
INFLUX_TOKEN    = "my-super-secret-token-123"
INFLUX_ORG      = "boiler_org"
INFLUX_BUCKET   = "boiler_data"
CHROMA_PATH     = "./chroma_db"
OLLAMA_MODEL    = "boiler-assistant"
# —————

class BoilerRAGPipeline:
    def __init__(self):
        print("▢ Initializing RAG pipeline...")

        # InfluxDB client
        self.influx = InfluxDBClient(
            url=INFLUX_URL, token=INFLUX_TOKEN, org=INFLUX_ORG
        )
        self.query_api = self.influx.query_api()

        # Embedding model (converts text to vectors for semantic search)
        self.embeddings = HuggingFaceEmbeddings(
            model_name="sentence-transformers/all-MiniLM-L6-v2"
        )

        # ChromaDB vector store
        self.vectorstore = Chroma(
            persist_directory=CHROMA_PATH,
            embedding_function=self.embeddings
        )

        # Fine-tuned LLM via Ollama
        self.llm = Ollama(model=OLLAMA_MODEL, temperature=0.1)

        print("▢ RAG pipeline ready")

    def get_realtime_context(self):
        """Fetch latest readings from InfluxDB as context string."""
        context_parts = []

        sensors = {
            "boiler_sensors": ["temperature", "pressure", "fuel_flow",
"water_level"],
            "chimney_sensors": ["flue_temp", "co2", "co", "draft"],
        }

        for measurement, sensor_list in sensors.items():
            device = "BOILER_001" if "boiler" in measurement else "CHIMNEY_001"
            for sensor in sensor_list:
                query = f"""
                from(bucket: "{INFLUX_BUCKET}")
                |> range(start: -5m)
                |> filter(fn: (r) => r["_measurement"] == "{measurement}")
                |> filter(fn: (r) => r["sensor"] == "{sensor}")
                |> filter(fn: (r) => r["_field"] == "value")
                |> last()
                """
                try:
                    tables = self.query_api.query(query)
                    for table in tables:
                        for record in table.records:
                            val = round(record.get_value(), 2)
                            status_tag = record.values.get("status", "NORMAL")
                            context_parts.append(

```

```

        f"{device}/{sensor}: {val} [{status_tag}]"
    )
    except Exception:
        pass

# Get recent faults
fault_query = f"""
from(bucket: "{INFLUX_BUCKET}")
  |> range(start: -1h)
  |> filter(fn: (r) => r["_measurement"] == "fault_events")
  |> filter(fn: (r) => r["_field"] == "message")
  |> last()
"""
try:
    tables = self.query_api.query(fault_query)
    for table in tables:
        for record in table.records:
            context_parts.append(
                f"RECENT FAULT: {record.values.get('fault_code')} "
                f"({record.values.get('severity')}) at
{record.get_time()}"
            )
    except Exception:
        pass

    return "\n".join(context_parts) if context_parts else "No recent data
available."

def index_fault_history(self):
    """Index fault history into ChromaDB for semantic retrieval."""
    print("🔍 Indexing fault history into ChromaDB...")
    query = f"""
from(bucket: "{INFLUX_BUCKET}")
  |> range(start: -7d)
  |> filter(fn: (r) => r["_measurement"] == "fault_events")
  |> filter(fn: (r) => r["_field"] == "message")
"""
    docs, metas, ids = [], [], []
    tables = self.query_api.query(query)
    for table in tables:
        for i, record in enumerate(table.records):
            text = (
                f"Fault: {record.values.get('fault_code')} "
                f"Severity: {record.values.get('severity')} "
                f"Time: {record.get_time()} "
                f"Message: {record.get_value()}"
            )
            docs.append(text)
            metas.append({"type": "fault", "time": str(record.get_time())})
            ids.append(f"fault_{i}")

    if docs:
        self.vectorstore.add_texts(docs, metadatas=metas, ids=ids)
        self.vectorstore.persist()
        print(f"✅ Indexed {len(docs)} fault records into ChromaDB")

def answer(self, question: str) -> str:
    """Answer a question using real-time data + RAG."""

    # 1. Get real-time context from InfluxDB
    realtime_context = self.get_realtime_context()

    # 2. Retrieve semantically similar past events from ChromaDB
    similar_docs = self.vectorstore.similarity_search(question, k=3)

```

```

        historical_context = "\n".join([doc.page_content for doc in
similar_docs])

        # 3. Build the full prompt
        prompt = f"""You are a boiler and chimney monitoring assistant.
Answer the question based ONLY on the real-time sensor data and fault history
below.
Do not use any general knowledge. If data is not available, say so.

=== CURRENT REAL-TIME READINGS ===
{realtime_context}

=== RELEVANT HISTORICAL EVENTS ===
{historical_context}

=== QUESTION ===
{question}

=== ANSWER ==="""

        # 4. Get LLM response
        response = self.llm(prompt)
        return response.strip()

if __name__ == "__main__":
    pipeline = BoilerRAGPipeline()

    # Index fault history (do this once, then incrementally)
    pipeline.index_fault_history()

    # Test it
    test_questions = [
        "What is the current boiler pressure?",
        "Are there any active faults?",
        "Is the boiler safe to operate?",
        "What was the last fault on the chimney?",
    ]

    for q in test_questions:
        print(f"\nQ: {q}")
        print(f"A: {pipeline.answer(q)}")

```

☐ Day 6 Checklist

- ☐ ChromaDB set up and fault history indexed
 - ☐ Ollama serving the fine-tuned model
 - ☐ RAG pipeline fetches real-time InfluxDB data as context
 - ☐ Test questions get answers grounded in actual sensor data
-
-

DAY 7 — FastAPI Chatbot + Full Integration

□ Goal

Build the final chatbot API and UI. Wire everything together. End-to-end: boiler → MQTT → InfluxDB → RAG → chatbot response.

Step 7.1 — FastAPI Chatbot Server

```
pip install fastapi uvicorn websockets
```

Create `api/chatbot_api.py`:

```
"""
FastAPI Chatbot API – exposes the RAG pipeline as REST + WebSocket endpoints.
"""

from fastapi import FastAPI, WebSocket
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel
import sys
sys.path.append("..")
from rag.rag_pipeline import BoilerRAGPipeline
import json
from datetime import datetime

app = FastAPI(title="Boiler & Chimney AI Chatbot", version="1.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

# Initialize RAG pipeline (takes ~30 seconds on first load)
pipeline = BoilerRAGPipeline()

class ChatRequest(BaseModel):
    question: str
    session_id: str = "default"

class ChatResponse(BaseModel):
    answer: str
    timestamp: str
    question: str

@app.get("/")
async def root():
    return {"status": "Boiler Chatbot API running", "version": "1.0"}

@app.get("/health")
async def health():
    return {"status": "healthy", "timestamp": datetime.utcnow().isoformat()}

@app.post("/chat", response_model=ChatResponse)
async def chat(request: ChatRequest):
    """Main chat endpoint – accepts a question, returns grounded answer."""
```

```

    answer = pipeline.answer(request.question)
    return ChatResponse(
        answer=answer,
        timestamp=datetime.utcnow().isoformat(),
        question=request.question,
    )

@app.get("/realtime")
async def realtime_data():
    """Return latest sensor readings as JSON."""
    context = pipeline.get_realtime_context()
    return {"data": context, "timestamp": datetime.utcnow().isoformat()}

@app.websocket("/ws/chat")
async def websocket_chat(websocket: WebSocket):
    """WebSocket endpoint for streaming chat responses."""
    await websocket.accept()
    try:
        while True:
            question = await websocket.receive_text()
            answer = pipeline.answer(question)
            await websocket.send_json({
                "answer": answer,
                "timestamp": datetime.utcnow().isoformat()
            })
    except Exception:
        pass

```

Run it:

```

cd api
uvicorn chatbot_api:app --host 0.0.0.0 --port 8000 --reload

```

API docs auto-generated at: <http://localhost:8000/docs>

Step 7.2 — Run the Complete System

Create `start_system.sh`:

```

#!/bin/bash
echo "🔧 Starting Boiler IoT + AI System..."

# 1. Start all Docker services
docker-compose up -d
sleep 10

# 2. Start simulators (background)
python simulators/boiler_simulator.py &
python simulators/chimney_simulator.py &

# 3. Start consumers (background)
python consumers/influx_consumer.py &
python consumers/fault_detector.py &

# 4. Start chatbot API
cd api && uvicorn chatbot_api:app --host 0.0.0.0 --port 8000

echo "🏁 All services running!"
echo "Grafana:      http://localhost:3000"
echo "EMQX:         http://localhost:18083"

```

```
echo "InfluxDB:      http://localhost:8086"
echo "Chatbot API:   http://localhost:8000"
echo "API Docs:      http://localhost:8000/docs"
```

Step 7.3 — Test the Full System

```
# Test the chatbot via curl
curl -X POST http://localhost:8000/chat \
  -H "Content-Type: application/json" \
  -d '{"question": "What is the current boiler pressure and is it safe?"}'

# Get real-time readings
curl http://localhost:8000/realtime
```

☐ Day 7 Checklist

- ☐ FastAPI server running at localhost:8000
 - ☐ /chat endpoint returns real-data-grounded answers
 - ☐ /realtime endpoint returns live sensor readings
 - ☐ WebSocket endpoint working for streaming chat
 - ☐ All services running simultaneously without errors
-

☐ Final Project Folder Structure

```
boiler-iot-system/
├── docker-compose.yml      ← All infrastructure (EMQX, InfluxDB, Grafana,
Kafka)
├── simulators/
│   ├── boiler_simulator.py ← Boiler sensor producer
│   └── chimney_simulator.py ← Chimney sensor producer
├── consumers/
│   ├── influx_consumer.py  ← MQTT → InfluxDB writer (Consumer #1)
│   └── fault_detector.py   ← MQTT → threshold rules → fault alerts
(Consumer #2)
├── exporters/
│   └── jsonl_exporter.py   ← InfluxDB → train.jsonl
├── data/
│   └── train.jsonl         ← Fine-tuning dataset
├── training/
│   └── fine_tune.py        ← LoRA fine-tuning script
└── models/
```

└─ boiler_llm/	← Saved fine-tuned model
└─ rag/	
└─ rag_pipeline.py	← LangChain + ChromaDB RAG
└─ chroma_db/	← Vector store (auto-created)
└─ api/	
└─ chatbot_api.py	← FastAPI chat endpoints
└─ start_system.sh	← One-command startup script
└─ requirements.txt	

Complete requirements.txt

```
paho-mqtt==1.6.1
influxdb-client==1.36.1
transformers==4.38.0
peft==0.8.0
datasets==2.17.0
accelerate==0.27.0
bitsandbytes==0.42.0
torch==2.2.0
langchain==0.1.9
langchain-community==0.0.24
chromadb==0.4.22
sentence-transformers==2.3.1
ollama==0.1.7
fastapi==0.109.2
uvicorn==0.27.1
websockets==12.0
```

☐ Key Concepts Summary

Concept	What it does in YOUR system
MQTT Publish	Boiler/chimney Python scripts send sensor readings
MQTT Subscribe	InfluxDB consumer, Kafka bridge, fault detector all receive data
EMQX Broker	Routes messages, handles 25K+ connections, provides dashboard
QoS 1	Sensor readings delivered at least once (safe default)
QoS 2	Fault events delivered exactly once (critical data)
InfluxDB	Stores every reading with timestamp, enables time-range queries
Grafana	Live visual dashboard — no code, just queries
Fault Detector	Consumer #2 — applies threshold rules, raises

Concept	What it does in YOUR system
	alerts, logs to InfluxDB
JSONL Exporter	Converts InfluxDB history into Q&A training pairs for LLM
LoRA	Fine-tunes 7B LLM using 0.1% of parameters — GPU-efficient
QLoRA	4-bit quantized LoRA — fits 7B model on 16GB GPU
RAG	Injects real-time sensor data into every LLM prompt
ChromaDB	Stores embeddings of fault history for semantic retrieval
FastAPI	Serves the chatbot as REST + WebSocket API

Built with: Python · MQTT · EMQX · InfluxDB · Grafana · Kafka · Mistral 7B · LoRA · LangChain · ChromaDB · FastAPI · Docker